

Python pathlib module

```
"""
=====
          PYTHON pathlib TUTORIAL
=====
```

pathlib is a built-in Python module for working with:

- Files
- Directories / folders
- File paths
- File extensions
- Creating folders
- Checking whether files exist
- Reading and writing files
- Moving and renaming files
- Deleting files/folders

For Data Engineering, pathlib is very useful for:

- Working with datasets
- Managing JSON/CSV/Parquet files
- Creating data directories
- Building file paths
- Processing files in pipelines
- Organizing input/output folders

Instead of:

```
"C:\\data\\input\\file.json"
```

you can use:

```
Path("data") / "input" / "file.json"
```

```
=====
"""
```

```
from pathlib import Path
```

```
# =====
# 1. IMPORT PATH
# =====
```

```
# Path is the main class from pathlib.
```

```
path = Path("data")
```

```
print(path)
```

```
# =====
# 2. CURRENT WORKING DIRECTORY
# =====
```

```
# Path.cwd() returns the directory where Python is
# currently running.
```

```
current_directory = Path.cwd()
```

```
print("\nCurrent directory:")
print(current_directory)
```

```

# =====
# 3. HOME DIRECTORY
# =====

# Path.home() returns the user's home directory.

home_directory = Path.home()

print("\nHome directory:")
print(home_directory)

# =====
# 4. CREATE PATHS
# =====

# You can create a path using Path().

file_path = Path("data/input/events.json")

print("\nFile path:")
print(file_path)

# =====
# 5. BUILD PATHS WITH /
# =====

# pathlib allows you to build paths using /.

data_dir = Path("data")
input_dir = data_dir / "input"
output_dir = data_dir / "output"
file_path = input_dir / "events.json"

print("\nBuilt paths:")
print(data_dir)
print(input_dir)
print(output_dir)
print(file_path)

# This is much better than manually writing:
# "data/input/events.json"

# =====
# 6. CREATE DIRECTORIES
# =====

# mkdir() creates a directory.

# exist_ok=True means:
# If the directory already exists, don't raise an error.

input_dir.mkdir(parents=True, exist_ok=True)
output_dir.mkdir(parents=True, exist_ok=True)

print("\nDirectories created.")

# parents=True allows pathlib to create parent
# directories if they don't already exist.

```

```

# Example:

# data/
#   input/
#
# pathlib can create both "data" and "input".

# =====
# 7. CHECK IF PATH EXISTS
# =====

print("\nDoes input directory exist?")
print(input_dir.exists())

# =====
# 8. CHECK FILE
# =====

print("\nIs input_dir a file?")
print(input_dir.is_file())

# =====
# 9. CHECK DIRECTORY
# =====

print("\nIs input_dir a directory?")
print(input_dir.is_dir())

# =====
# 10. FILE NAME
# =====

file_path = Path("data/input/events.json")

print("\nFile name:")
print(file_path.name)

# Output:
# events.json

# =====
# 11. FILE SUFFIX / EXTENSION
# =====

print("\nFile extension:")
print(file_path.suffix)

# Output:
# .json

# =====
# 12. FILE STEM
# =====

# stem = filename without extension

print("\nFile stem:")
print(file_path.stem)

```

```

# Output:
# events

# =====
# 13. PARENT DIRECTORY
# =====

print("\nParent directory:")
print(file_path.parent)

# Output:
# data/input

# =====
# 14. MULTIPLE SUFFIXES
# =====

compressed_file = Path("data/events.json.gz")

print("\nSuffix:")
print(compressed_file.suffix)

print("\nAll suffixes:")
print(compressed_file.suffixes)

# suffix:
# .gz

# suffixes:
# ['.json', '.gz']

# =====
# 15. ABSOLUTE PATH
# =====

relative_path = Path("data/input/events.json")

print("\nRelative path:")
print(relative_path)

print("\nAbsolute path:")
print(relative_path.absolute())

# resolve() can also produce an absolute path.

print("\nResolved path:")
print(relative_path.resolve())

# =====
# 16. CREATE A FILE
# =====

file_path = input_dir / "events.txt"

# write_text() creates the file if it doesn't exist.

file_path.write_text("Hello Data Engineering!")

print("\nFile created:")
print(file_path)

```

```

# =====
# 17. READ A FILE
# =====

content = file_path.read_text()

print("\nFile content:")
print(content)

# =====
# 18. WRITE MULTIPLE LINES
# =====

file_path = input_dir / "data.txt"

file_path.write_text(
    "Alice\n"
    "Bob\n"
    "Charlie\n"
)

print("\nCreated data.txt")

# =====
# 19. READ LINES
# =====

content = file_path.read_text()

lines = content.splitlines()

print("\nLines:")

for line in lines:
    print(line)

# =====
# 20. WRITE JSON FILE
# =====

# pathlib handles the FILE PATH.
# json handles the JSON DATA.

import json

json_file = input_dir / "events.json"

events = [
    {
        "id": 1,
        "name": "Laptop",
        "value": 1200
    },
    {
        "id": 2,
        "name": "Phone",
        "value": 800
    }
]

with json_file.open("w") as file:
    json.dump(events, file, indent=4)

print("\nJSON file created:")
print(json_file)

```

```

# =====
# 21. READ JSON FILE
# =====

with json_file.open("r") as file:
    data = json.load(file)

print("\nJSON data:")

for event in data:
    print(event)

# =====
# 22. LIST FILES IN DIRECTORY
# =====

print("\nFiles inside input directory:")

for item in input_dir.iterdir():
    print(item)

# =====
# 23. ONLY FILES
# =====

print("\nOnly files:")

for item in input_dir.iterdir():
    if item.is_file():
        print(item)

# =====
# 24. ONLY DIRECTORIES
# =====

print("\nOnly directories:")

for item in data_dir.iterdir():
    if item.is_dir():
        print(item)

# =====
# 25. GLOB
# =====

# Find all JSON files.

print("\nJSON files:")

for file in input_dir.glob("*.json"):
    print(file)

# =====
# 26. FIND ALL CSV FILES
# =====

print("\nCSV files:")

for file in input_dir.glob("*.csv"):
    print(file)

```

```

# =====
# 27. RECURSIVE SEARCH
# =====

# rglob() searches inside subdirectories too.

print("\nAll JSON files recursively:")

for file in data_dir.rglob("*.json"):
    print(file)

# Example structure:

# data/
#   input/
#     events.json
#     customers.json
#   raw/
#     2026/
#       events.json
#
# rglob() can find all of them.

# =====
# 28. RENAME FILE
# =====

old_file = input_dir / "data.txt"
new_file = input_dir / "customers.txt"

if old_file.exists():
    old_file.rename(new_file)

print("\nRenamed file:")
print(new_file)

# =====
# 29. MOVE FILE
# =====

# rename() can also be used to move a file.

source = input_dir / "customers.txt"

destination = output_dir / "customers.txt"

if source.exists():
    source.rename(destination)

print("\nMoved file:")
print(destination)

# =====
# 30. DELETE FILE
# =====

temporary_file = input_dir / "temporary.txt"

temporary_file.write_text("Temporary data")

print("\nTemporary file exists:")
print(temporary_file.exists())

temporary_file.unlink()

```

```

print("Temporary file exists after deletion:")
print(temporary_file.exists())

# =====
# 31. FILE SIZE
# =====

if json_file.exists():

    size = json_file.stat().st_size

    print("\nJSON file size:")
    print(size, "bytes")

# =====
# 32. FILE METADATA
# =====

if json_file.exists():

    info = json_file.stat()

    print("\nFile metadata:")
    print("Size:", info.st_size)
    print("Modified:", info.st_mtime)
    print("Created:", info.st_ctime)

# =====
# 33. FILE NAME MANIPULATION
# =====

file = Path("events.json")

print("\nName:", file.name)
print("Stem:", file.stem)
print("Suffix:", file.suffix)

# =====
# 34. CHANGE FILE EXTENSION
# =====

csv_file = Path("data/events.csv")
json_version = csv_file.with_suffix(".json")

print("\nOriginal:")
print(csv_file)

print("New extension:")
print(json_version)

# =====
# 35. CHANGE FILE NAME
# =====

file = Path("data/events.json")
new_name = file.with_name("customers.json")

print("\nOriginal:")
print(file)

print("New name:")
print(new_name)

```



```

# =====
# 36. PATH PARTS
# =====

file = Path("data/raw/2026/events.json")

print("\nPath parts:")

for part in file.parts:
    print(part)

# Output:

# data
# raw
# 2026
# events.json

# =====
# 37. PATH PARENT
# =====

file = Path("data/raw/2026/events.json")

print("\nParent:")
print(file.parent)

print("\nParent's parent:")
print(file.parent.parent)

# =====
# 38. RELATIVE PATH
# =====

path = Path("data/raw/events.json")

try:
    relative = path.relative_to("data")

    print("\nRelative path:")
    print(relative)

except ValueError:
    print("Path is not inside data directory.")

# =====
# 39. RESOLVE
# =====

path = Path("data/input/events.json")

print("\nResolved path:")
print(path.resolve())

# =====
# 40. PATH + LOOP
# =====

print("\nProcess all JSON files:")

for file in input_dir.glob("*.json"):
    print("Processing:", file)

```

```

# Here you could:
#
# read JSON
# validate data
# transform data
# write output
# move processed file

# =====
# 41. DATA ENGINEERING EXAMPLE
# =====

"""
Imagine your pipeline has:

data/
  raw/
  processed/
  archive/

You receive JSON files into raw/.

Your pipeline:

    raw
    |
    v
  process JSON
    |
    v
  processed
    |
    v
  archive

pathlib is perfect for managing these paths.
"""

base_dir = Path("data")
raw_dir = base_dir / "raw"
processed_dir = base_dir / "processed"
archive_dir = base_dir / "archive"
raw_dir.mkdir(parents=True, exist_ok=True)
processed_dir.mkdir(parents=True, exist_ok=True)
archive_dir.mkdir(parents=True, exist_ok=True)

print("\nData Engineering directories created:")
print("RAW:", raw_dir)
print("PROCESSED:", processed_dir)
print("ARCHIVE:", archive_dir)

# =====
# 42. FIND JSON FILES IN RAW
# =====

print("\nRaw JSON files:")

for file in raw_dir.glob("*.json"):
    print("Found:", file)

```

```

# =====
# 43. DATA PIPELINE EXAMPLE
# =====

"""
A simple file-processing pipeline:

1. Find JSON files
2. Read them
3. Process them
4. Write output
5. Move original file to archive
"""

for file in raw_dir.glob("*.json"):

    print("\nProcessing:", file)

    # Read
    with file.open("r") as f:
        data = json.load(f)

    print("Records:", len(data))

    # Create output filename
    output_file = processed_dir / file.name

    # Write processed data
    with output_file.open("w") as f:
        json.dump(data, f, indent=4)

    print("Written:", output_file)

    # Archive original
    archive_file = archive_dir / file.name

    # Uncomment when you actually want to move files.
    #
    # file.rename(archive_file)

    print("Archive location:", archive_file)

# =====
# 44. IMPORTANT pathlib METHODS
# =====

"""
Path()

cwd()
home()

exists()
is_file()
is_dir()

mkdir()
iterdir()
glob()
rglob()

read_text()
write_text()

open()

rename()
unlink()

```

```
absolute()
resolve()

stat()

name
stem
suffix
suffixes
parent
parts

with_name()
with_suffix()
relative_to()
"""
```

```
# =====
# 45. IMPORTANT DATA ENGINEERING PATTERN
# =====
```

```
"""
Instead of hardcoding:

    "C:/Users/Youssef/project/data/input/events.json"
```

Use:

```
base_dir = Path("data")

input_dir = base_dir / "input"

file = input_dir / "events.json"
```

This makes your code:

```
- cleaner
- easier to maintain
- easier to move
- cross-platform
- easier to reuse
"""
```

```
# =====
# 46. PRACTICE TASKS
# =====
```

```
"""
TASK 1
-----
Create:
```

```
data/
  raw/
  processed/
  archive/
```

```
TASK 2
-----
Create:
```

```
data/raw/events.json
```

TASK 3

Write this JSON:

```
[
    {"id": 1, "value": 100},
    {"id": 2, "value": 200}
]
```

TASK 4

Read events.json.

TASK 5

Print:

```
    filename
    extension
    parent directory
```

TASK 6

Find all .json files inside data/raw.

TASK 7

Move events.json from:

data/raw/

to:

data/processed/

TASK 8

Rename:

events.json

to:

events_processed.json

TASK 9

Find all JSON files recursively inside data/.

TASK 10

Build this path using pathlib:

```
data/
  output/
    2026/
      09/
        events.json
```

TASK 11

Create 5 JSON files:

```
events_1.json
events_2.json
events_3.json
events_4.json
events_5.json
```

TASK 12

Loop through all JSON files and print:

Processing <filename>

TASK 13

Move processed files into:

data/archive/

TASK 14

Build a simple pipeline:

```
raw
↓
processed
↓
archive
```

Use pathlib for ALL path operations.

"""

```
# =====
# FINAL SUMMARY
# =====
```

```
print("\n" + "=" * 60)
```

```
print("pathlib tutorial completed!")
```

```
print("=" * 60)
```

```
print(
    """
```

For Data Engineering, remember these first:

```
    from pathlib import Path

    base = Path("data")

    input_dir = base / "input"

    output_dir = base / "output"

    input_dir.mkdir(parents=True, exist_ok=True)

    file = input_dir / "events.json"

    file.exists()

    file.is_file()
```

```
file.read_text()
file.write_text("data")
file.glob("*.json")
file.rglob("*.json")
file.rename(new_path)
file.unlink()
file.name
file.stem
file.suffix
file.parent
file.parts
pathlib = Python's tool for managing file paths.
"""
)
```